

Next month

What Blu-ray or DVD player should you buy? Find out in next month's issue

5870 Eyefinity Edition

AMD quietly launches their Radeon HD 5870 Eyefinity Edition, with 6 mini DisplayPort connects, and 2 GB of GDDR5 memory



The DX11 Unigine benchmark called Heaven. The intricate level of detail on the rocks and the stone steps mimics reality, courtesy Tessellation. You can expect to see games with such visual detail when DX11 is used

is hardly useful for games, but is definitely needed for simulations and number crunching tasks like genetic sequencing, or intense math computations. Not only did Microsoft simplify programmer's lives a bit by adding certain object-oriented programming concepts to SM 5.0 but it was designed to reduce the load on memory as well as increase the efficiency of its usage.

Multithreaded Rendering: Working out those muscle-bound CPUs

Ever wonder really high-clocked dual core CPUs are faster than slower quad cores when it comes to gaming? CPUs take on workloads in the form of threads – four cores, four threads at a time and so on. Games aren't exactly designed to assign tasks to CPUs that are designed to work on individual threads. While games might tax the mightiest GPU, they generate no more than a handful of threads to keep the CPU busy – video, physics and AI being good examples. After all, how many threads can one application (a single game) create? Therefore, your quad core CPU is left largely unutilised. For the better part, the game will thrash one or two cores, while the others will sit pretty, because these GPU-generated threads have to be processed as a whole and cannot be broken down further. DX11 was designed with the growing number of cores on today's CPUs in mind and it tries to harness them better

by sending threads to idle cores. Once processed, these threads can be dynamically inserted into the rendering pipeline when the GPU so desires. When one considers that six core CPUs are already available, and eight core CPUs should be available from as early as next year, this feature makes sense as well as smacks of Microsoft truly having thought of the future.

This technique quite obviously improves performance by getting more work done at the same time by outsourcing work better. While the performance increment isn't going to be dramatic and it may not be noticeable some of the time, this performance tweak costs virtually nothing in terms of resources – freebies are most welcome.

JARGON BUSTER

DirectX (DX): DX is an API for Windows-based systems for handling multimedia related tasks including game programming.

Patches: Are a new type of primitive, like vertices and pixels that define the segment of a plane that is to be tessellated.

Tessellation: simplifies complex wireframes, reducing the GPU load for the next process in the pipe and also thereby reducing memory footprint of the application.

Rasterization: the process of taking a 3D image as described using vector graphics and converting it into a raster image consisting of pixels for output on a display.

Tessellation: Over-hyped or hyper feature?

In the processing world nothing comes free. Extra performance needs extra powerful hardware, getting a task done quicker means adding extra processing power and so on. To get better visual quality at a minimal loss of FPS is the stuff game developers dream of. The software component of the Tessellator, (built into DX11), is a little different from AMDs proprietary implementation of the same on their DX10.1 cards. Quite simply the Tessellator takes rough shapes (primitives) and breaks them into smaller parts. It can then construct immensely detailed shapes from the re-assembled smaller parts. For example a sphere can be formed from a simple cube. The DX11 Tessellator is not programmable but the input to it, and output from it, are. These two stages are the Hull Shader (HS) and Domain Shader (DS). The HS comes just before the Tessellator and its input is primitives and control points for these primitives. It uses the control points to determine how the process of tessellation needs to be done. The actual Tessellator is just a fixed-function block – it will break up the primitives handed to it based on instructions provided by the HS. Because of its fixed-function nature there is no coding required for tessellation. After this the task passes to the Domain Shader; this programmed block outputs displacement maps / geometry. This geometry can head off for rasterization directly, or be further processed by the Geometry Shader.

Theoretically, the Geometry Shader, which is a programmable block, can handle tessellation as well, but the benefit of using a dedicated, fixed-function block is sheer performance – the Geometry Shader also has a whole lot of other tasks on its plate, which is why the dedicated Tessellator came into existence.

Direct X Compute: Do more, so much more

The Compute Shader enables generic algorithms on what is a render-centric API. With DX11 Compute we see more general-purpose code as well as an increased set of operations that can be